

Week 5

23_Microservices_Architecture_Introduction

Microservices Architecture

- Need for microservices
- Service-oriented development
- Independent services

Features

- Modular design
- Independent deployment
- Easy maintenance
- Faster updates

Simple Meaning

- Dividing a large application into smaller independent services.

Description

Microservices architecture is a software development approach in which a large application is divided into multiple small and independent services. Each service is responsible for performing a specific business function and communicates with other services using APIs. This architecture improves flexibility because services can be developed, tested, and deployed separately. It enables organizations to update individual components without affecting the entire application. Microservices are widely used in modern cloud-native applications because they support scalability and faster software delivery. Companies such as Netflix and Amazon use microservices to manage large-scale systems efficiently.

24_Monolithic_vs_Microservices

Comparison

- Monolithic Architecture
- Microservices Architecture

Features

- Single application vs multiple services
- Tight coupling vs loose coupling
- Centralized vs distributed deployment

Simple Meaning

- Understanding the differences between traditional and modern application architectures.

Description

A monolithic application contains all functionalities such as user interface, business logic, and database access within a single codebase. Any modification requires rebuilding and redeploying the entire application. In contrast, microservices architecture divides the application into smaller services that can operate independently. Monolithic systems are easier to start with but become difficult to maintain as they grow larger. Microservices offer better scalability, fault isolation, and development flexibility. Organizations choose microservices when applications need to handle large traffic and frequent updates. Understanding both architectures helps developers select the most suitable design for a project.

25_Microservices_Advantages

Advantages

- Scalability
- Isolation
- Agility
- Independent deployment

Features

- Better fault tolerance
- Faster development cycles
- Improved resource utilization

Simple Meaning

- Benefits of using microservices in modern software development.

Description

Microservices provide several advantages over traditional application architectures. Each service can be scaled independently according to workload requirements, reducing infrastructure costs. Service isolation ensures that the failure of one service does not affect the entire application. Development teams can work on different services simultaneously, increasing productivity and reducing development time. Microservices support continuous integration and continuous deployment (CI/CD), enabling faster software releases. They also allow organizations to use different technologies and programming languages for different services. These benefits make microservices a popular choice for large and complex applications.

26_API_Gateway

API Gateway

- Central entry point
- Request routing
- Load balancing
- Service communication

Features

- Authentication and authorization
- Traffic management
- Monitoring and logging
- Security enforcement

Simple Meaning

- A single access point that manages requests to multiple microservices.

Description

An API Gateway acts as an intermediary between clients and microservices. Instead of communicating directly with multiple services, clients send requests to the API Gateway. The gateway routes requests to the appropriate service and returns responses to the client. It provides important functions such as authentication, authorization, load balancing, caching, logging, and monitoring. API Gateways improve security by protecting internal services from direct exposure. They also simplify client-side development by providing a unified interface for accessing multiple services. API Gateways are essential components in modern microservices-based applications.

27_Docker_Compose_Introduction

Docker Compose

- Multi-container management
- Service orchestration
- Application deployment

Features

- Simplified configuration
- Automated container startup
- Easy networking between services
- Consistent environments

Simple Meaning

- A tool used to run and manage multiple Docker containers together.

Description

Docker Compose is a tool that helps developers define and manage multi-container Docker applications using a single YAML configuration file. Instead of starting containers individually, Docker Compose allows all services to be launched with a single command. It automatically creates networks, manages dependencies, and configures communication between containers. Docker Compose is commonly used for deploying applications that include databases, backend services, and frontend components. It ensures consistency across development, testing, and production environments. By reducing manual configuration, Docker Compose improves productivity and simplifies application deployment processes.